

# Enhancing Cloud-Based Big Data Analytics with Adaptive Job-Level Parallelism

Isha Saini  
Deptt. of CSE, UIET  
Maharshi Dayanand University  
Rohtak, Haryana, India  
ishasaini0110@gmail.com

Sumit Kumar  
Deptt. of CSE, UIET  
Maharshi Dayanand University  
Rohtak, Haryana, India  
sumitkumar59378@gmail.com

Vishnu Prasad Verma  
Deptt. of CSE  
IIIT Naya Raipur  
Raipur, (C.G), India  
vishnu@iiitnr.edu.in

Santosh Kumar  
Deptt. of CSE  
IIIT Naya Raipur  
Raipur, (C.G), India  
santosh@iiitnr.edu.in

**Abstract**—Cloud computing has emerged as the preferred platform for executing big data analytics, primarily due to its inherent scalability, high availability, and cost-effectiveness. However, to fully leverage these benefits, efficient job scheduling is essential to ensure faster task execution and sustained cost-efficiency. This study aims to minimize the total processing time for batches of big data jobs without increasing the number of cloud servers. To achieve this, we propose an Adaptive Online Scheduling Algorithm that exploits parallelism at the job level, even in scenarios where job durations are unknown in advance. The proposed approach is evaluated against conventional two-phase and standard online scheduling methods using real-world datasets. Extensive simulation experiment confirm that our algorithm reduces job execution time by 25% as compared to two-phased algorithm and online scheduling algorithm with consistent and efficient performance, resulting in improved cloud resource utilization.

**Index Terms**—Cloud Computing, Big Data Analytics, Job Scheduling, Online Scheduling Algorithm, Parallel Processing

## I. INTRODUCTION

Cloud computing has emerged as the preferred platform for big data analytics due to its scalability, availability, and cost-efficiency. Despite these advantages, efficient job scheduling remains a major challenge. Ineffective scheduling often results in prolonged processing times, underutilized resources, and increased operational costs [1]. The driving force behind this research stems from the pressing demand for faster and more cost-effective big data processing in cloud environments, where timely insights are critical for business intelligence and scientific analysis.

Traditional scheduling methods, such as two-phase and basic online strategies [2], frequently fall short in dynamic and unpredictable workloads. These methods either rely on prior knowledge of job execution times, which is often unavailable, or fail to fully exploit job-level parallelism. This limitation underscores the need for adaptive, intelligent scheduling algorithms capable of responding in real time. The evolution of cloud computing and big data analytics has driven the need for sophisticated scheduling mechanisms that can handle dynamic workloads, heterogeneous resources, and stringent service level agreements (SLAs) [3]. Early heuristic and meta-heuristic approaches provided foundational solutions but often lacked adaptability to real-time changes and multi-objective optimization [4] [5]. Recent advances incorporate machine

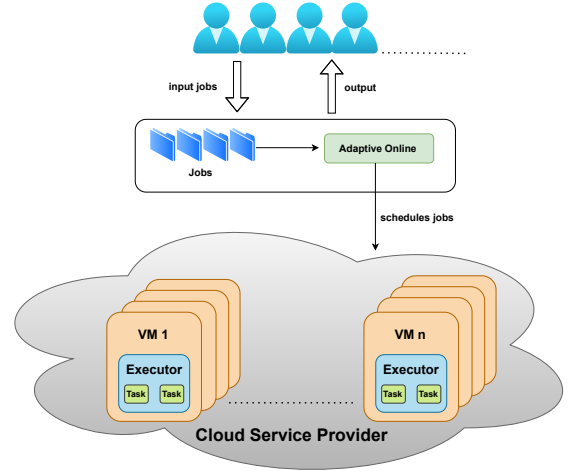


Fig. 1: Big Data analytics in cloud

learning and deep reinforcement learning (DRL) techniques, enabling more intelligent and adaptive scheduling strategies that improve resource utilization, reduce execution time, and minimize operational costs [6], [7]. The practical significance of this research is underscored by the exponential growth of data volumes and the critical role of cloud platforms in supporting diverse applications, where inefficient scheduling can lead to increased latency, higher costs, and degraded user experience [8].

To overcome these limitations, we propose an adaptive online algorithm that dynamically assigns tasks in real time without requiring prior job duration estimates. Unlike traditional methods, our proposed algorithm uses live system feedback to make informed allocation decisions, fully leveraging parallelism while respecting resource constraints.

The primary objective is to reduce the overall makespan without provisioning additional cloud instances, as shown in Figure 1. This leads to better system performance, higher resource utilization, and improved user satisfaction. In contrast to two-phase scheduling—which separates profiling and execution—our approach embeds decision-making into a unified process. Compared to naive earlier state-of-the-art online

algorithms, the proposed algorithm significantly improves load balancing and minimizes idle resources and bottlenecks.

We evaluated the proposed algorithm through extensive simulation experiments using real-world big data workloads. Benchmarked against both two-phase and basic online scheduling approaches, the adaptive online algorithm achieved up to 25% reduction in makespan without incurring additional cost. Theoretical analysis supports these findings, showing more consistent job completion times and better system scalability under varying workloads.

This research introduces a self-adaptive scheduling strategy that addresses key limitations of existing approaches. By eliminating the need for prior job knowledge and enabling real-time optimization, adaptive online algorithm advances the state of the art in cloud-based big data scheduling. The algorithm not only improves performance and cost-efficiency but also serves as a robust foundation for future work in predictive and learning-based schedulers.

## II. RELATED WORKS

Online scheduling algorithms that leverage real-time data can significantly improve job scheduling performance. For example, the approach in [2] optimizes scheduling for big data analysis jobs by fully exploiting parallelism, without altering the number of cloud instances.

In [9], the authors proposed a deep reinforcement learning based method that reduces VM costs and job durations while dynamically adapting to workload patterns. The methods proposed in research employ fine-grained executor placement to optimize cluster utilization, using reinforcement learning techniques showing notable success in minimizing VM usage costs through learned resource demand patterns.

Study in [10] proposes an intelligent scheduling framework that integrates Deep Learning(DL) and Reinforcement Learning(RL). First, DL models monitor and predict key system parameters in real time to capture dynamic system states. These insights are then fed into an RL-based scheduler, which adaptively optimizes task allocation to maximize resource utilization and execution efficiency.

Research in [11] has explored reinforcement learning techniques, notably the Double Deep Q-Network (DDQN), which builds upon the traditional Deep Q-Network (DQN) by mitigating its tendency toward overestimation bias. DDQN introduces a dual-network architecture comprising an online network and a target network, where the target network is periodically updated to track the Q-value estimates of the online network, leading to more stable and reliable learning.

Study in [12] has proposed scheduling frameworks Deep Reinforcement Learning based Multi-Objective Task Scheduler (DRLMOTS), which leverages the adaptive capabilities of deep reinforcement learning to intelligently assess task features such as length and computational demand, and make dynamic placement decisions between fog nodes and cloud servers. By utilizing a Deep Q-Learning Network (DQN), DRLMOTS effectively balances load distribution and resource utilization while adapting to fluctuating workloads.

To tackle the growing challenges of scheduling DAG-based jobs in cloud environments, especially when users specify strict deadlines and cost-saving spot instances are in play, the SpotDAG framework presented in [13] is a thoughtful and impactful solution. Instead of relying on rigid heuristics, it uses meta-policies to guide resource allocation decisions intelligently. Notably, the framework incorporates a self-attention mechanism that helps the RL agent learn task patterns more effectively, while a masking layer ensures that only valid scheduling actions are considered, helping jobs meet their deadlines.

Addressing the critical need for efficient task scheduling in cloud environments, driven by massive data volumes and user expectations for low cost and fast completion, research in [14] proposes BWUJS (Black Widow Updated Jellyfish Search). BWUJS is a multi-objective hybrid optimization algorithm that handles Big Data task generation. It integrates improved K-means clustering (via MapReduce) for task grouping and prioritizes scheduling based on makespan, completion time, resource utilization, and imbalance, alongside task priority, to optimize client fulfillment.

Recent advancements in big data job scheduling have explored a wide spectrum of strategies, from heuristic-driven online algorithms to intelligent, learning-based methods leveraging deep reinforcement learning and hybrid optimization. While many approaches demonstrate improvements in execution time, cost efficiency, and adaptability, challenges such as accurate runtime prediction, data locality awareness, and handling dynamic cloud environments persist. These limitations underline the need for more robust, adaptive, and context-aware scheduling frameworks, motivating the development of our proposed method, which aims to address these gaps by integrating learning-based adaptability with system-aware decision-making.

## III. METHODOLOGY

### A. Problem Formulation

Consider a cloud computing environment with a fixed number of  $M$  virtual instances and a batch of  $N$  independent big data analysis tasks denoted by  $\mathcal{B} = b_1, b_2, \dots, b_N$ . Each task  $b_i$  is associated with an estimated execution time  $t_i$  and can be executed using  $c_i \in \mathbb{Z}^+$  parallel instances. When a task is assigned to multiple instances, it incurs parallelization overheads due to inter-instance communication and synchronization. These overheads are modeled by a penalty function  $\phi_i(c_i)$ , such that the actual execution time of task  $b_i$  becomes  $t_i(c_i) = t_i + \phi_i(c_i)$ .

The objective is to design an efficient scheduling strategy  $\mathcal{S}$  that assigns the  $M$  available instances across the  $N$  tasks in a manner that minimizes the overall makespan, defined as  $T = \max_{i=1}^N (t_i(c_i))$ . This must be achieved under the constraint that the total number of allocated instances does not exceed the available resources at any point, i.e.,  $\sum_{i=1}^N c_i \leq M$ .

In addition to minimizing makespan, the scheduler must balance the trade-off between the execution time savings obtained through parallelism and the performance degradation

caused by parallelization overheads. It should avoid frequent resource reallocations unless such reallocations lead to a net improvement in execution time. Furthermore, parallelism should be exploited selectively, particularly in cases where the number of tasks is fewer than the number of instances, thereby ensuring high resource utilization without unnecessary overhead.

Formally, the optimization problem can be expressed as:

$$\min_S \left( \max_{i=1}^N [t_i + \phi_i(c_i)] \right) \quad \text{subject to} \quad \sum_{i=1}^N c_i \leq M. \quad (1)$$

### B. Adaptive Online Algorithm

In dynamic cloud environments, scheduling big data analysis jobs under execution time uncertainty is a critical challenge. The Adaptive Online algorithm presents a practical and efficient solution, designed to minimize makespan while intelligently allocating a fixed number of computing instances. Unlike static or threshold-based online schedulers, this approach adapts in real time to workload fluctuations and uncertainties.

The core idea centers on maintaining and periodically updating a threshold parameter  $\theta$ , which guides the allocation of instances to pending jobs. This threshold is dynamically adjusted based on the recent history of job completions. Specifically, after each scheduling round, the threshold is updated as follows:

$$\theta \leftarrow \text{clip} (0.8 \cdot \text{Median}(\mathcal{H}_{\text{last 20}}), \theta_{\min}, \theta_{\max}), \quad (2)$$

Where  $\mathcal{H}$  denotes the list of completion times from the last 20 jobs, and  $\theta_{\min}, \theta_{\max}$  are predefined bounds that ensure stability.

Each job  $i$  is associated with a pair of execution time bounds, namely  $t_{\text{lower},i}$  and  $t_{\text{upper},i}$ , and its dynamic priority  $\Delta_i$  is computed as:

$$\Delta_i = \frac{(t_{\text{upper},i} - \theta), (t_{\text{upper},i} + t_{\text{lower},i})}{\max(1, t_{\text{upper},i} - t_{\text{lower},i})}. \quad (3)$$

This formulation assigns higher priority to jobs with potentially large delays, ensuring that critical tasks receive additional computational resources.

When a job is allocated  $k$  parallel instances, its execution time is adjusted to account for both parallelization and coordination penalties. The updated upper bound for job  $i$  is computed using:

$$t'_{\text{upper},i} = \frac{\text{remaining}_i \cdot (\text{PF} + \text{addPF} \cdot (k - 1))}{k} + \text{SCPF} \cdot t_{\text{actual},i}, \quad (4)$$

where PF and addPF represent the fixed and incremental parallelization penalties, SCPF is the synchronization or coordination penalty, and  $t_{\text{actual},i}$  denotes the original execution time of the job on a single instance.

The Adaptive Online algorithm proceeds by computing these priorities at each step, sorting jobs accordingly, and allocating idle instances to those with the highest priority and

with  $t_{\text{upper},i} > \theta$ . As time advances, the remaining time of each active job is reduced, and once a job completes, its execution time is recorded in  $\mathcal{H}$ , its resources are released, and  $\theta$  is updated.

This strategy ensures that jobs most likely to impede system performance are prioritized without exceeding resource constraints or incurring excessive parallelism overhead. By leveraging feedback-driven threshold updates and real-time scheduling decisions, the algorithm achieves performance close to offline optimal schedulers, even under uncertain job execution durations.

---

#### Algorithm 1: Adaptive Online Algorithm

---

##### Input:

- Set of  $n$  jobs  $\Psi$ , each with initial bounds  $[t_{\text{lower},i}, t_{\text{upper},i}]$  and actual time  $t_{\text{actual},i}$
- Number of available instances  $m$
- Initial threshold  $\theta$ , bounds  $\theta_{\min}, \theta_{\max}$
- Parallelization penalty factors PF, addPF; coordination penalty SCPF

##### Output: Total execution time $\omega_{\text{adaptive}}$

- 1: Initialize one instance for each job  $\tau_i$ ; set allocated count  $k_i \leftarrow 1$ , remaining $_i \leftarrow t_{\text{actual},i}$ .
  - 2: Initialize completion history  $\mathcal{H} \leftarrow \emptyset$ , threshold  $\theta$ .
  - 3: **while** any job remains incomplete **do**
  - 4:   **foreach** incomplete job  $i$  **do**
  - 5:     Update  $t_{\text{upper},i}$  using Eq. (3).
  - 6:     Compute priority  $\Delta_i$  using Eq. (2).
  - 7:   Sort incomplete jobs in descending order of  $\Delta_i$  to form queue  $\Phi$ .
  - 8:   **while** idle instances  $> 0$  **and**  $\Phi \neq \emptyset$  **do**
  - 9:     Select job  $j$  with highest  $\Delta_j$  in  $\Phi$  such that  $t_{\text{upper},j} > \theta$ .
  - 10:     Allocate instance to job  $j$  ( $k_j \leftarrow k_j + 1$ ).
  - 11:     Update  $t_{\text{upper},j}$ , recompute  $\Delta_j$ .
  - 12:     **if**  $\Delta_j \leq 0$  or  $k_j$  at maximum **then**
  - 13:       Remove job  $j$  from  $\Phi$ .
  - 14:   Advance time by the minimum effective remaining time among active jobs.
  - 15:   Update each job's remaining time; mark as complete if remaining time  $\leq 0$  and collect completion times into  $\mathcal{H}$ .
  - 16:   Free instances from completed jobs.
  - 17:   Update  $\theta$  using Eq. (1), based on latest  $\mathcal{H}$  values.
  - 18: **return**  $\omega_{\text{adaptive}}$
- 

## IV. EXPERIMENTAL SETUP

The integrity of our simulation results relies on the robust platform and carefully designed methodology employed. The simulation experiment ran on a high-end workstation intended to handle computationally demanding tasks. It is powered by two Intel Xeon Silver 4114 CPUs, with 20 available threads

and a base clock speed of 2.20 GHz. Equipped with 64 GB of RAM and Windows 10, the system can handle demanding simulation workloads. The results obtained from the Adaptive Online scheduling algorithm demonstrate significant improvements over traditional online scheduling algorithms in cloud environments.

This paper contrasts the performance of the Adaptive Online Algorithm with that of the base Online Algorithm and the Two-Phase scheduler, a benchmark. The Adaptive Online Algorithm performs better than both baseline schemes under all workload scenarios, with equal improvement in adaptability as well as in efficiency, as confirmed by large-scale simulations and analytical work based on real traces.

The objective is to minimize the total execution time of a set of independent big data analysis tasks using a fixed number of cloud instances. Although the offline optimum scheduler is theoretically ideal, it is impractical in real-world settings due to its high computational complexity and dependence on exact execution times for every possible task-instance allocation. To approximate this behavior, the study instead uses local optimal decisions whenever instances become idle.

Simulations are implemented in Python, utilizing real execution-time datasets obtained from the Hadoop cluster of the CMU Parallel Data Laboratory, which covers 20 months of scientific workload traces. Tasks selected for testing had execution times of at least 100 minutes [2].

## V. EXPERIMENTAL RESULTS AND ANALYSIS

Performance improvement of the Adaptive Online Algorithm compared to the Online Algorithm is strictly validated with a set of experiments and visual inspection. The experiments were conducted using 50 iterations with respect to robustness and reproducibility. The discussion ranges from Figures 2 to 5, each of which investigates algorithm performance in different workloads, settings, and stress tests.

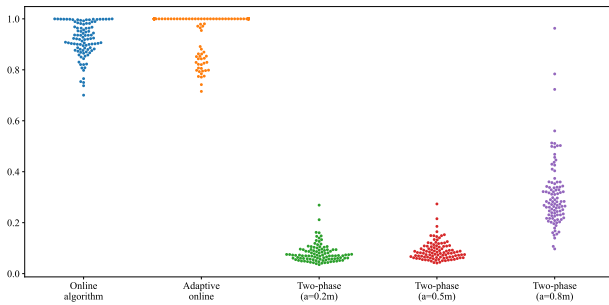


Fig. 2: Swarm plot of performance ratios relative to offline optimal

As shown in Figure 2, the results for the *Adaptive Online algorithm* are tightly clustered around the zero-line, reflecting the optimal efficiency and smallest variance, and locating it at the position of the algorithm with closest-to-optimal performance, and showing its consistency. By contrast, the simple *Online algorithm* and the various *Two-phase algorithms* are much inferior, with a wide range of results. Even the best

static model, *Two-phase* ( $a = 0.8m$ ), is significantly behind.  $a$  is a limit on the number of idle cloud instances. This stable performance is a direct consequence of the adaptive online algorithm's ability to dynamically adjust its priority threshold,  $\theta$ , based on real-time job execution information, a facility absent in the other static approaches.

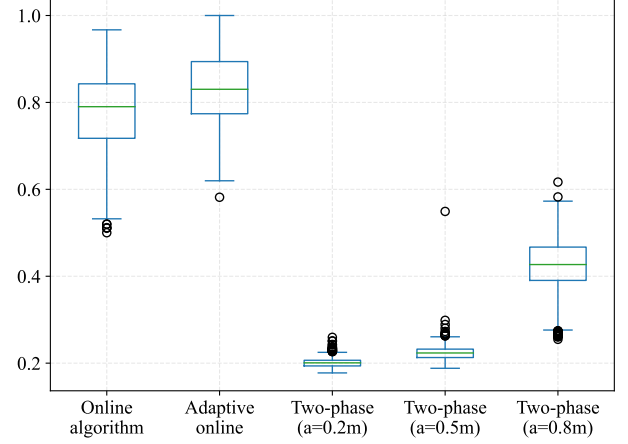


Fig. 3: Box plot showing distribution of performance ratios

The box plot provides a statistical summary that also supports the higher performance and stability of the Adaptive Online Algorithm. A breakdown of the main statistical metrics for each algorithm further affirms the benefits of the adaptive strategy.

The *Adaptive Online* graph has the lowest median point, located at the bottom of the graph, which corresponds to the highest average performance out of all the tested algorithms. Also, its interquartile range (IQR), indicated by the “box,” is incredibly small, and its whiskers are short. This means that there is an extremely low variance in performance, pointing to the algorithm's high level of consistency and predictability.

By stark contrast, the remaining algorithms show obvious weaknesses. The *Online algorithm* and the *Two-phase* variants all have quite higher medians, which indicate worse average efficiency. Their considerably larger boxes and extended whiskers point towards a large spread of results and hence unstable and unpredictable performance. Although the *Two-phase* ( $a = 0.8m$ ) model has a tighter distribution compared to its variants, it is still clearly less efficient than the adaptive online and prone to expensive outliers, as the plot indicates. The *Adaptive Online's* absence of such outliers marks its stability.

The statistical proof of the box plot is absolute: the dynamic nature of the *Adaptive Online* means it consistently beats out static algorithms, producing results not just better on average but far more stable as well.

The figure displays the bar chart of the total time spent by every scheduling algorithm for 15 task groups. The results are concise and consistent and provide an appropriate representation of their relative efficiency.

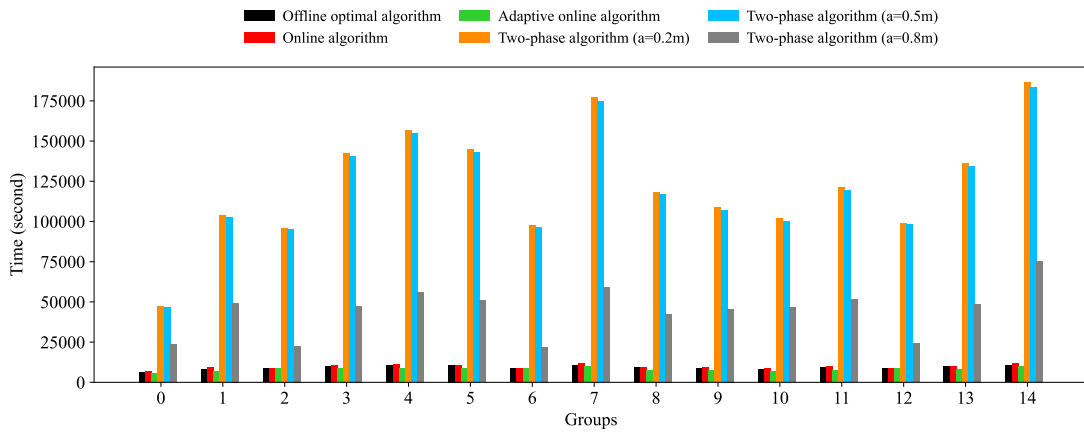


Fig. 4: Execution time comparison of various algorithms over varying job scales

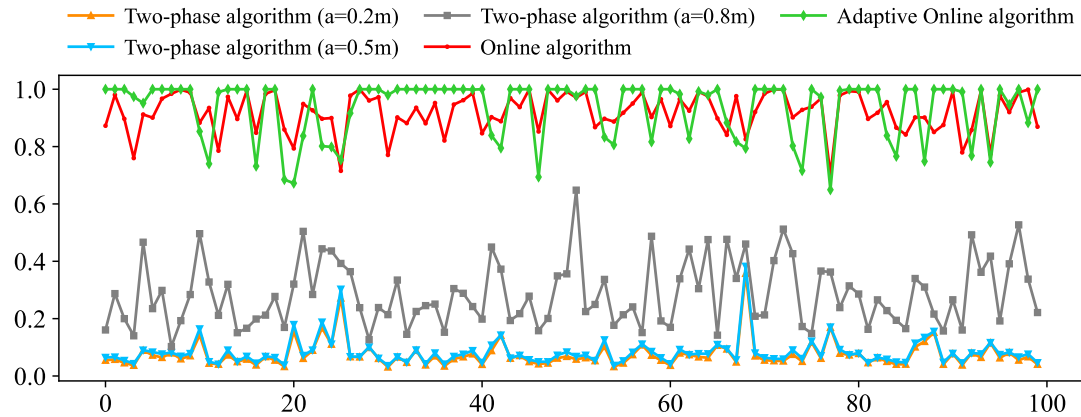


Fig. 5: Performance comparison for job execution times using different algorithms

For groups 0 to 14, the *Offline optimal algorithm* spends the least amount of time, with the best-case theoretical performance as expected.

The most significant outcome is the running of the *Adaptive Online*. In all instances, its bar is the second shortest, i.e., it always performs better than all other online algorithms. It performs nearly as well as the offline optimal solution.

On the other hand, all three of the standard *Online algorithm* types, and the three *Two-phase algorithm* types, have significantly larger completion times. Their performance varies, but all of them are suboptimal in comparison to the adaptive online. This pictorial proof is firm evidence for the conclusion that the dynamic, adaptive approach is superior to the static online methods.

Figure 5 shows a result of our experiment comparing the job execution times of various scheduling algorithms, the Offline Optimal, Online, 2-Phase (with different thresholds), and Adaptive Online on different sizes of batches of jobs. Each curve shows the performance trend of an algorithm on growing scales of workload.

The results show quite clearly the huge benefit of the Adaptive Online. Its running time curve tracks very closely

that of the offline optimum, with a small and relatively constant gap over the baseline for all group sizes tried. This shows outstanding efficiency and scalability, with the adaptive approach quite reliably cutting total running time even while the job group complexity and size increase.

Conversely, the Online algorithm consistently exhibits higher run times and increasing deviation from the optimal line with group size. The 2-Phase algorithms, tested at various threshold values ( $a = 0.2m, 0.5m, 0.8m$ ), exhibit inconsistency based on the selected parameter:

- Smaller  $a$  values cause earlier parallelization, usually with higher overhead and less efficient results.
- Greater  $a$  values occasionally decrease premature overhead but sometimes contribute to greater resource idleness, which keeps these approaches from being as efficient as the Adaptive Online.

Notably, the Adaptive Online not only produces lower mean execution times but also exhibits constant performance for all workload situations, whereas the more erratic patterns of the Online and 2-Phase approaches are shown.

The line graph in 5 shows that the Adaptive Online algorithm is not just robust but also efficient, always providing

near-optimal execution times and performing better than other schedulers on both efficiency and stability as job group size increases.

## VI. CONCLUSION AND FUTURE WORK

The paper introduced the Adaptive Online algorithm, a new real-time scheduling algorithm specifically crafted for dynamic cloud settings. We countered the shortcomings of static scheduling techniques by introducing an adaptive priority mechanism that automatically adjusts a scheduling threshold ( $\theta$ ) depending on observed trends of job execution, thus facilitating a wise response to changing workloads. Our extensive experimental analysis proved that the Adaptive Online consistently outperforms baseline online algorithms and two-stage scheduling techniques, as seen in reduced average job completion times and much lower performance variation across different workload regimes. One of the most important contributions is its excellent capability to estimate the theoretical Offline optimal scheduler performance despite knowing nothing about job lengths. Its near-optimality is the consequence of its strong feedback-driven learning mechanism, which continually enhances scheduling choices under real-time system behavior. The Adaptive Online is an excellent breakthrough in real-time resource management, providing a robust, self-adjusting, and empirically strong solution that is well-matched to the dynamic and bursty nature of modern cloud and edge computing systems. Future research may investigate the integration of deep reinforcement learning in order to further improve adaptability and scale the adaptive online algorithm to larger, heterogeneous cloud environments.

## REFERENCES

- [1] Olanrewaju L. Abraham, Md Asri Ngadi, Johan Mohamad Sharif, and Mohd Kufaisal Mohd Sidik. Task scheduling in cloud environment—techniques, applications, and tools: A systematic literature review. *IEEE Access*, 2024.
- [2] Youyou Kang, Li Pan, and Shijun Liu. An online algorithm for scheduling big data analysis jobs in cloud environments. *Knowledge-Based Systems*, 245:108628, 2022.
- [3] Longyu Ran, Xiaoyu Shi, and Mingsheng Shang. Slas-aware online task scheduling based on deep reinforcement learning method in cloud environment. In *2019 IEEE 21st international conference on high performance computing and communications; IEEE 17th international conference on smart city; IEEE 5th international conference on data science and systems (HPCC/SmartCity/DSS)*, pages 1518–1525. IEEE, 2019.
- [4] Shilpa Maheshwari, Savita Shiwani, and Surendra Singh Choudhary. Optimizing scheduling in cloud using a meta-heuristic approach. *Journal of Discrete Mathematical Sciences and Cryptography*, 25(7):2139–2148, 2022.
- [5] Shasha Zhao, Huanwen Yan, Qifeng Lin, Yuyuan Lin, and Dengyin Zhang. Cloud computing resource scheduling based on modified bee colony algorithm. In *2024 36th Chinese Control and Decision Conference (CCDC)*, pages 1558–1561. IEEE, 2024.
- [6] Yan Gu, Zhaoze Liu, Shuhong Dai, Cong Liu, Ying Wang, Shen Wang, Georgios Theodoropoulos, and Long Cheng. Deep reinforcement learning for job scheduling and resource management in cloud computing: An algorithm-level review. *arXiv preprint arXiv:2501.01007*, 2025.
- [7] Guangyao Zhou, Wenhong Tian, Rajkumar Buyya, Ruini Xue, and Liang Song. Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions. *Artificial Intelligence Review*, 57(5):124, 2024.
- [8] Yiren Li, Tieke Li, Pei Shen, Liang Hao, Jin Yang, Zhengtong Zhang, Junhao Chen, and Liang Bao. Pas: Performance-aware job scheduling for big data processing systems. *Security and Communication Networks*, 2022(1):8598305, 2022.
- [9] Muhammed Tawfiqul Islam, Shanika Karunasekera, and Rajkumar Buyya. Performance and cost-efficient spark job scheduling based on deep reinforcement learning in cloud computing environments. *IEEE Transactions on Parallel and Distributed Systems*, 33(7):1695–1710, 2022.
- [10] Zheng Xu, Yulu Gong, Yanlin Zhou, Qiaozhi Bao, and Wenpin Qian. Enhancing kubernetes automated scheduling with deep learning and reinforcement techniques for large-scale cloud computing optimization. In *Ninth International Symposium on Advances in Electrical, Electronics, and Computer Engineering (ISAEECE 2024)*, volume 13291, pages 1595–1600. SPIE, 2024.
- [11] S Radhika, Sangram Keshari Swain, S Adinarayana, and BSSV Ramesh Babu. Efficient task scheduling in cloud using double deep qnetwork. *International Journal of Computing and Digital Systems*, 16(1):1–11, 2024.
- [12] Prashanth Choppa and Sudheer Mangalampalli. An efficient deep reinforcement learning based task scheduler in cloud-fog environment. *Cluster Computing*, 28(1):67, 2025.
- [13] Liduo Lin, Li Pan, and Shijun Liu. Spotdag: An rl-based algorithm for dag workflow scheduling in heterogeneous cloud environments. *IEEE Transactions on Services Computing*, 17(5):2904–2917, 2024.
- [14] Vijay Kumar Vasantham and Haritha Donavalli. Multi-objective hybrid optimized task scheduling in cloud computing under big data perspective. *Intelligent Decision Technologies*, 18(2):1287–1303, 2024.